

## CS1810 Notes Oct 1: Affine Gap Alignment

- Definition: Inputs:  $\langle X, Y, \alpha, \mu, \gamma, \tau \rangle$ 
  - $X, Y$  = strings of length  $m, n$  with characters indexed by  $i, j$  respectively
  - $\alpha$  = match score
  - $\mu$  = mismatch penalty
  - $\gamma$  = gap opening penalty
  - $\tau$  = gap extension penalty (single-letter gap penalty)

Outputs: An alignment which maximizes the score:

$\alpha(\# \text{ matches}) - \mu(\# \text{ mismatches}) - \gamma(\# \text{ gap clusters}) - \tau(\# \text{ single-letter gaps})$

- Definitions: Auxiliary Data Structures
  - Matrices:  $V, G, E, F$ 
    - $V$  = the best-score matrix
    - $G$  = the match-mismatch matrix
    - $E$  = the X-gap matrix
    - $F$  = the Y-gap matrix
- An Example:
  - $X = \text{ATCGGC}$
  - $Y = \text{AGC}$
  - $\alpha = 2$  (score = +2)
  - $\mu = 1$  (penalty = -1)
  - $\gamma = 2$  (penalty = -2)
  - $\tau = 1$  (penalty = -1)

### Algorithm:

$V(0,0) \leftarrow 0$

For  $i \leftarrow 1$  to  $m$  do:

$V(i,0) = E(i,0) \leftarrow -\gamma - i \cdot \tau$

End

For  $j \leftarrow 1$  to  $n$  do:

$V(0,j) = F(0,j) \leftarrow -\gamma - j \cdot \tau$

End

For  $i \leftarrow 1$  to  $m, j \leftarrow 1$  to  $n$  do:

$G(i,j) \leftarrow \begin{cases} V(i-1, j-1) + \alpha & \text{if } x_i = y_j \\ V(i-1, j-1) - \mu & \text{if } x_i \neq y_j \end{cases}$

$E(i,j) \leftarrow \max \begin{cases} E(i, j-1) - \tau, \\ V(i, j-1) - \gamma - \tau \end{cases}$

$F(i,j) \leftarrow \max \{ F(i-1, j) - \tau,$

$$\{ V(i-1, j) - \gamma - \tau$$

$$V(i,j) \leftarrow \max \{ G(i,j), E(i,j), F(i,j) \}$$

End

There are quadratically many steps,  $m * n$  steps...

(using  $\alpha = 2$ ,  $\gamma = 2$ ,  $\mu = 1$ ,  $\tau = 1$ )

V: (X across the top, Y down the side; the letter after each score = which matrix the max came from)

|   |    | A    | T      | C      | G    | G      | C    |
|---|----|------|--------|--------|------|--------|------|
|   | 0  | -3   | -4     | -5     | -6   | -7     | -8   |
| A | -3 | 2_G  | -1_F   | -2_F   | -3_F | -4_F   | -5_F |
| G | -4 | -1_E | 1_G    | -2_G/F | 0_G  | -1_G   | -4_F |
| C | -5 | -2_E | -2_G/E | 3_G    | 0_F  | -1_G/F | 1_G  |

- The score: (C,C)  $\rightarrow$  (G,G)  $\rightarrow$  (A,G)  $\rightarrow$  (A,C)  $\rightarrow$  (A,T)  $\rightarrow$  (A,A)  $\rightarrow$  0
- A T C G G C
- A - - - G C

Score +1 (3 matches = +6, 1 gap cluster = -2, 3 single-letter gaps = -3)

The powerpoint keeps going...

The algorithm continues like this until all the matrices are filled... we'll skip ahead stopping at some interesting intermediate states which cover new branches in the algorithm.

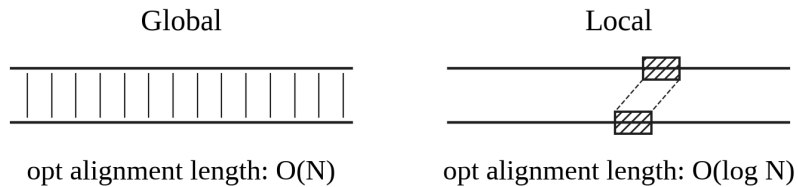
Traceback: use the backpointers in our V matrix to reconstruct our alignment

At each position we can recover the single-valued alignment...

- Examples of alignment: The shazam algorithm, speech recognition (hidden markov models)...

From the whiteboard:

- Random biosequences: AAB, BAC, BBC, BCA
  - random model:  $\text{Prob}(A) = 4/12$ ,  $\text{Prob}(B) = 5/12$ ,  $\text{Prob}(C) = 3/12$
- Alignment of 2 random sequences (length N):
  - Global: length of the opt alignment is  $O(N)$
  - Local: length of the opt alignment is  $O(\log N)$



- “How is it that it stops at length  $O(\log N)$  for the OPT local alignment?”
  - Answer: the very special scoring scheme → Dayhoff LOD score, log-ratio...
  - on random sequences a pair of letters scores negative on average, so making a local alignment longer just loses score

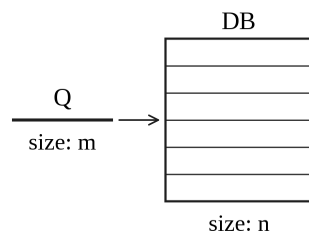
- Karlin-Altschul Statistics (1990)
  - → only for ungapped alignments
  - 5 axioms:

|       |                                                                             |
|-------|-----------------------------------------------------------------------------|
| Ax. 1 | Scoring matrix needs to have at least one positive score                    |
| Ax. 2 | The expected score of the scoring matrix needs to be negative               |
| Ax. 3 | Letters of the random sequences are independent and identically distributed |
| Ax. 4 | Sequences are infinitely long                                               |
| Ax. 5 | Alignments don't contain gaps...                                            |

- ↳ Karlin-Altschul Equation:

$$E = K * m * n * e^{(-\lambda * S)}$$

- $E$  = number of alignments expected by chance for alignments of score  $S$  (or better)
- database search (BLAST): a database of size  $n$  and a query sequence of size  $m$
- $m * n$  = the search space;  $K$  and  $\lambda$  are constants that come from the scoring matrix



- back to the lady tasting tea... if  $E$  is really small, the alignment probably isn't just chance

- Dayhoff LOD score:

- $\text{LOD}(\text{pair of amino acids}) = \log_2(\text{obs. freq} / \text{expected freq in the random model})$
- $s_{ij} = \log_2(q_{ij} / (p_i * p_j))$ 
  - $q_{ij}$  = how often i and j are actually seen paired,  $p_i * p_j$  = how often they'd pair by chance

|         |                                           |
|---------|-------------------------------------------|
| lod = 0 | obs. freq = expected freq                 |
| lod > 0 | pairing i and j in an alignment is common |
| lod < 0 | pairing unlikely...                       |

- Ungapped pairs of max score:

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| A | C | G | T | T | A |
| A | G | C | T | A | A |